VeriLM: A Commit-and-Audit Protocol for Open-Weight LLM Inference

Federico Carrone, Diego Kingston, Mariano Nicolini, Pedro Fontana, Manuel Puebla
LambdaClass

Preprint

Abstract — Users of open-weight LLMs currently have no technical mechanism to verify which model actually ran, or whether the output was altered by the provider or an intermediary. We present VeriLM, a system for auditing open-weight LLM inference that provides computational integrity and verifiable provenance: a verifier can confirm that a response is tied to the claimed model weights and serving configuration, detecting both silent downgrades (for example, serving 8B in place of 70B) and post-commitment tampering (the commitment scheme binds all activations, so any modification after receipt issuance is detectable). The core insight is that public weights permit audit rather than proof-system-heavy verification. VeriLM deploys as a sidecar-style audit layer over existing serving stacks. In the default deployment path, it leaves model weights unchanged, requires no kernel modification, and shifts the main cost to rare audits rather than the normal token-generation path. Each response carries a 100-byte receipt; only audited responses open traces. The verification architecture has five layers: (1) exact shell verification — cryptographic Freivalds checks on all INT8 weight matmuls plus exact recomputation of requantization, RoPE, RMSNorm, and SiLU; (2) KV provenance — hash-committed per-token K, V history with exact commitment binding and statistical correctness from sampled shell checks on earlier tokens; (3) cross-layer consistency — opening multiple layers on the same token creates algebraic coupling through the residual stream, forcing any fake attention to stay consistent across all opened layers; (4) attention replay — the verifier recomputes attention from shell-verified Q and committed prefix K,V in FP64, quantizes the result, and compares against the committed output; and (5) statistical coverage on unopened tokens and layers. This decomposition gives exact verification on the shell (model identity is caught unconditionally, $\leq 1/2^{32}$), statistical provenance on prefix state, and approximate replay on the attention interior — limited by FP16↔{}FP64 mismatch and the KV sampling boundary. Because receipts are small, VeriLM supports continuous random auditing at low amortized cost, keeping providers under persistent technical accountability.

1 Introduction

Open-weight LLM inference presents a trust problem: the client sends a prompt to a provider who claims to run a specific model (e.g., Llama 70B), but the client has no way to verify that the provider actually used those weights. The economic incentive to cheat is clear — serving a smaller or more aggressively quantized model reduces compute costs while the client pays the same price.

Existing approaches to verifiable inference fall into two categories. Cryptographic proof systems and specialized verifiable-inference systems such as SafetyNets [1] and zkCNN [2] can prove arbitrary or restricted computations, but impose overhead that makes real-time LLM inference impractical. Trusted execution environments (TEEs) provide attestation [3] but require hardware trust assumptions and limit deployment flexibility.

VeriLM takes a different approach: a lightweight sidecar protocol that runs alongside unmodified GPU inference. The key insight is that INT8 quantized inference consists of operations that are either deterministic or canonically recomputable, and can be verified exactly. The only non-deterministic component is FP16/BF16 attention, which the protocol constrains from both sides without requiring exact reproduction.

The protocol provides five layers of verification with different guarantee types, ranging from exact cryptographic checks on weight matrices to statistical sampling of prefix history.

2 System Model

2.1 Threat Model

The adversary controls the inference server. They may swap model weights, modify attention computation, fabricate intermediate activations, or lie about earlier context. The adversary does not know which responses will be audited or which tokens within them will be challenged. In the intended deployment, the client's verifier software automatically and randomly audits a small fraction of responses (e.g., 1–5%). Because every response is committed before the provider knows whether it will be challenged, the provider must behave as if any response could be audited.

The client holds a verifier key (~ 25 MB for Llama 70B) derived from the public model weights and a secret random vector. The client audits responses directly — no trusted third party sees the conversation. The attester / verifier / relying-party terminology here follows standard remote-attestation usage [3].

VeriLM provides interactive, client-held verification. The client who holds the verifier key checks the computation directly. The protocol does not produce a transferable proof that third parties can verify offline — this is a deliberate tradeoff for low overhead. Audit openings contain intermediate activations from which the conversation content can be reconstructed; this is why the client audits themselves rather than delegating to a third party.

2.2 Design Principles

VeriLM is designed around three constraints:

1. **No serving-path changes.** The provider runs unmodified GPU inference. The only addition is a tracing layer that captures intermediates alongside normal execution.
2. **Minimal per-response overhead.** Each response carries a 100-byte receipt (two Merkle roots, a deployment manifest hash, and a token count). Most responses are never audited.
3. **Client-side verification.** The client performs all verification using CPU-feasible operations (dot products, hash

checks, and — for attention replay — matrix arithmetic scaling as $O(n^2)$ in sequence length).

Protocol guarantees at a glance.

1. **Exact.** Opened-layer model identity ($\leq 1/2^{32}$ false-accept) and commitment binding (hash-immutable after receipt).
2. **Statistical.** Prefix KV correctness under sampling: $P(\text{catch}) = 1 - (1 - m/n)^k$.
3. **Approximate.** Attention replay under the FP16↔FP64 requantization corridor (Section 6).

Symbol	Meaning
R_W	Merkle root over model weights (public identity)
R_T	Merkle root over trace intermediates
R_{KV}	Merkle root over per-token K, V history
M	Deployment manifest hash
N	Token count in response
c, ℓ	Challenged token positions; opened layers per token
k	Sampled prefix positions for KV provenance
τ	Acceptance threshold for attention replay

Table 1: Notation

3 Verification Layers

VeriLM's verification is structured in five layers. This section defines the taxonomy and key terms; the full procedure is specified in Section 4.

The **shell** is the non-attention path through each transformer layer: seven INT8 weight matrix multiplications (W_q, W_k, W_v, W_o, W_{gate}, W_{up}, W_{down}), **requantization bridges** (the i32 → i8 conversions between consecutive matmul stages), RoPE, RMSNorm, and SiLU. The shell includes nonlinear operations (RMSNorm, SiLU), but all shell operations are deterministic or canonically recomputable. Its exactness is a composition: cryptographic checks (Freivalds [4]) on the weight matmuls, plus deterministic or canonical recomputation on the bridge operations.

Attention (QK^T, softmax, αV) is computed in FP16/BF16, which is not bit-reproducible across hardware — this is the only non-exact component. The five layers are:

1. **Shell verification (exact).** Cryptographic Freivalds checks ($\leq 1/2^{32}$ false-accept) on all weight matmuls, plus deterministic recomputation of requantization bridges, RoPE, RMSNorm, and SiLU.
2. **KV provenance (statistical).** Merkle-committed per-token K,V history; sampled positions are shell-verified. Binding is exact; correctness of unsampled positions depends on sampling rate.
3. **Cross-layer consistency (structural).** Opening multiple layers on the same token creates algebraic coupling through the residual stream — fake attention must stay consistent across all opened layers.
4. **Attention replay (approximate).** The verifier recomputes attention from shell-verified Q and committed prefix K,V in FP64, quantizes to INT8, and compares. Limited by FP16↔FP64 mismatch.
5. **Unopened positions (none).** No direct verification. Deterrence from the provider not knowing which responses will be audited or which tokens challenged.

Listing 1 traces the full forward pass for one output token. Each operation is labeled with its verification type: Freivalds-checked weight matmuls, canonically recomputable bridge operations, and the single non-exact point — FP16 attention. The only non-replayable provider-side state is the per-layer `attn_out_i8` and its quantization scale; everything below that point replays exactly or canonically from the stored values, the transcript, and the public weights.

```
residual = embedding[token_id]          [exact lookup]

for each layer:
  x_norm = RMSNorm(residual)            [canonical]
  x_i8, s = quantize(x_norm)            [exact]

  q_i32 = W_q @ x_i8                    [Freivalds]
  k_i32 = W_k @ x_i8                    [Freivalds]
  v_i32 = W_v @ x_i8                    [Freivalds]

  q = RoPE(dequant(q_i32, ...), pos)    [canonical]
  k = RoPE(dequant(k_i32, ...), pos)    [canonical]
  v = dequant(v_i32, ...)               [canonical]

  attn = softmax(q @ k^T / sqrt(d)) @ v [non-exact]
  attn_out_i8, sa = quantize(attn)      [STORED]

  o_i32 = W_o @ attn_out_i8             [Freivalds]
  residual += dequant(o_i32, ...)       [canonical]

  x_norm = RMSNorm(residual)            [canonical]
  x_i8, s = quantize(x_norm)            [exact]

  gate_i32 = W_gate @ x_i8              [Freivalds]
  up_i32   = W_up   @ x_i8              [Freivalds]
  x = SiLU(dequant(gate_i32))           [canonical]
      * dequant(up_i32)
  x_i8, s = quantize(x)                 [exact]
  down_i32 = W_down @ x_i8              [Freivalds]
  residual += dequant(down_i32, ...)    [canonical]

x_norm = RMSNorm(residual)              [canonical]
x_i8, s = quantize(x_norm)              [exact]
logits_i32 = LM_head @ x_i8             [Freivalds]
token = sample(dequant(logits_i32))     [transcript]
```

Listing 1: Annotated forward pass for one output token. Each operation is labeled with its verification type. The only non-exact stage is FP16 attention and the quantization that depends on it; storing `attn_out_i8` and its scale bridges the gap.

4 The Protocol

4.1 Phase 0: Setup

Public identity. A Merkle root [5] R_W is computed over all weight matrices of the published checkpoint. This root is the model's public identity — anyone can recompute it from the published weights.

Verifier key generation. Each verifier independently generates, for each of the 7 matrix types (W_q, W_k, W_v, W_o, W_{gate}, W_{up}, W_{down}), a secret random vector r_j. For each layer i, the verifier precomputes $v_j^{(i)} = r_j^T \times W_j^{(i)}$ — one matrix-vector multiply per matrix per layer, performed once. The resulting verifier key is ~ 25 MB for Llama 70B. After precomputation, the verifier deletes the weights. The r_j vectors are the verifier's secret; if the prover learns them, it can forge passing checks.

4.2 Phase 1: Commitment

The provider runs inference normally. The serving path is unchanged except for a tracing layer that captures intermediates alongside execution. For every token at every layer (both prefill and decode), the provider records:

- The INT8 input to each of the 7 weight matrices
- The INT32 accumulator output ($W_{\text{i8}} \times x_{\text{i8}}$, exact)
- The requantized INT8 values at each bridge
- The post-attention INT8 output
- The per-tensor quantization scales at each requantization bridge

After generation completes, the provider builds two Merkle trees [5]. The trees are separate because they serve different access patterns: shell verification (Step 2) opens a few positions from R_T, while KV provenance (Step 3) opens the full prefix from R_{KV}. A single tree would force opening all intermediates at every prefix position just to extract the KV values.

- R_T (trace tree): over all intermediates at all tokens. The provider cannot change any activation after committing.
- R_{KV} (KV tree): over per-token K, V state across all layers. The provider cannot retroactively rewrite earlier tokens' context. This tree allows efficient opening of the full prefix KV without opening the complete trace at every position.

A deployment manifest binds everything outside the forward pass:

$$M = H(\text{tokenizer_hash} \parallel R_W \parallel \text{quant_hash} \parallel \text{sampling_params} \parallel \text{eos_policy} \parallel \text{system_prompt_hash}) \tag{1}$$

The provider returns the response plus a 100-byte receipt (R_T, R_{KV}, M, N). Most responses are never audited. The receipt is the only overhead on the normal serving path.

4.3 Phase 2: Verification

The client decides to audit a response. The provider does not know which responses will be challenged or which tokens within them will be opened.

4.3.1 Step 1: Challenge

The verifier selects c random token positions from the N-token response and, for each, chooses ℓ layers to open. Opening multiple layers on the same token is preferred — this enables cross-layer consistency checks (Step 4).

4.3.2 Step 2: Shell Verification

For each challenged token t at each opened layer i, the provider opens the INT8 inputs x_{i8}, INT32 accumulators z_{i32}, per-tensor quantization scales, the residual stream values at the layer boundaries, and the post-attention output attn_out_i8 — all with Merkle proofs against R_T. The verifier performs four checks:

1. **Freivalds on each weight matrix.** For each of the 7 matrices, the verifier checks $v_j^{(i)} \cdot x \stackrel{?}{=} r_j^T \cdot z$. Each check is two dot products — $O(n)$. If the prover used wrong weights, false-accept probability is $\leq 1/2^{32}$ per matrix.
2. **Requantization bridges (exact).** The verifier recomputes the i32 $\rightarrow$ i8 conversion elementwise from the opened accumulators and quantization scales. This is deterministic integer arithmetic. Without this check, the prover could pass Freivalds (correct $r \cdot z$) but feed fabricated i8 values into the next stage.
3. **SiLU (exact).** INT8 inputs take 256 possible values. The verifier checks every element against a precomputed 256-entry lookup table.
4. **RoPE and RMSNorm.** RoPE is recomputed from the position index — deterministic integer-scaled arithmetic. RMSNorm is canonically recomputed from the opened residual stream values; the result is verified in the quantized output space (the recomputed value must requantize to the same i8 as the committed value).

After this step, the verifier has trusted Q_t, K_t, V_t (as requantized INT8 values with verified RoPE) for the challenged token.

4.3.3 Step 3: KV Provenance

Attention at token t requires the full prefix $K_{1..t}$, $V_{1..t}$. Shell verification gives trusted values for the challenged token but not the prefix.

For each opened layer i, the provider opens the K, V values at all prefix positions $j < t$ with Merkle proofs against R_{KV}. The verifier then:

1. Verifies the Merkle proofs — confirming these are the values the prover committed in Phase 1.
2. Randomly samples k earlier token positions $j_1, ..., j_k < t$.
3. For each sampled position j_s, runs full shell verification at layer i (Freivalds on all 7 matrices, requantization, RoPE, RMSNorm, SiLU), producing independently verified $K_{j_s}^{(i)}$, $V_{j_s}^{(i)}$.
4. Compares these shell-verified values against the corresponding entries opened from R_{KV}.

If the prover tampered with m out of n prefix positions and the verifier samples k:

$$P(\text{catch}) = 1 - (1 - m/n)^k \tag{2}$$

Commitment binding is exact (hash collision resistance). Correctness of unsampled positions is statistical — the prover risks the verifier sampling any corrupted position.

4.3.4 Step 4: Cross-Layer Consistency

There is no separate proof object here; the consistency check comes from opening multiple layers and verifying the committed bridges between them. When the verifier opens layers L and $L+1$ on the same token, fake attention at layer L must produce a post-attention output that, after requantization and W_o, feeds into layer $L+1$'s RMSNorm consistently with the committed trace. Both sides of this boundary are shell-verified, so the adversary cannot fabricate a consistent bridge without matching the committed values exactly. The more layers opened, the tighter the constraint. In full-audit mode (all layers opened), fake attention at every layer must stay mutually consistent through the entire residual stream.

4.3.5 Step 5: Attention Replay

This step combines the trusted Q_t from Step 2 with the commitment-verified prefix $K_{1..t}$, $V_{1..t}$ from Step 3. The verifier recomputes attention independently in FP64. Since the KV cache stores post-RoPE values, the verifier dequantizes directly and computes:

1. Scores: $Q_t \cdot K_j / \sqrt{d}$ for all prefix positions j
2. Softmax over scores
3. Weighted sum: $\sum \alpha_j \times V_j$

The verifier quantizes the result to INT8 using the quantization scales opened from the trace (committed in R_T) and compares element-wise against the committed post-attention output. A response passes if at least a fraction τ of INT8 elements agree; τ must be calibrated empirically by measuring the FP16$\leftrightarrow$FP64 requantization corridor on real model activations (see Section 6).

This pins the attention output to a specific computation over the committed values. Without it, the adversary could fabricate a plausible-looking but wrong attention output — a

local-window approximation, suppressed attention to specific positions, or a cheaper model's attention pattern — as long as it satisfied cross-layer consistency.

Limitations. The replay proves consistency with the *committed* prefix, not necessarily with true execution at every earlier token. Prefix KV values are commitment-verified (they match R_{KV}) but only statistically anchored to real computation via the sampled shell checks in Step 3. The replay also cannot match the GPU's FP16 attention exactly — the verifier's FP64 reference will differ slightly near quantization bucket boundaries.

4.4 Summary

Table 2 summarizes the verification method applied to each operation in the forward pass.

Operation	Verification
Input embedding	Table lookup (exact)
W_q, W_k, W_v (INT8)	Freivalds
Requantize i32 → i8	Exact recomputation
RoPE on Q, K	Exact recomputation
Attention	Replay + cross-layer
W_o (INT8)	Freivalds
RMSNorm	Canonical recomputation
W_{gate}, W_{up} (INT8)	Freivalds
SiLU $\odot$ up	256-entry LUT + exact
W_{down} (INT8)	Freivalds
LM head	Freivalds

Table 2: Per-layer verification methods

4.5 Audit Walkthrough: One Token, Two Layers

Audit token $t = 47$ at layers 12 and 13 of a 70B model, sampling $k = 8$ prefix positions.

1. **Challenge.** Verifier sends $(t = 47, \{12, 13\})$.
2. **Shell.** Provider opens trace at both layers from R_T. Verifier runs Freivalds on all 7 matrices per layer (28 dot products total), recomputes all bridges. Yields trusted Q_{47}, K_{47}, V_{47} at both layers.
3. **KV provenance.** Provider opens $K_{1..46}$, $V_{1..46}$ at both layers from R_{KV}. Verifier picks 8 random earlier positions, runs full shell verification at each, confirms committed K, V match.
4. **Cross-layer.** Post-attention output at layer 12 feeds into layer 13's RMSNorm — both sides shell-verified, so boundary must match exactly.
5. **Replay.** At each opened layer, the verifier recomputes attention in FP64 from shell-verified Q_{47} and the commitment-verified prefix. Quantizes to INT8, compares against committed `attn_out_i8`. Passes if $\geq \tau$ elements agree at both layers.

4.6 Data Lifecycle

Table 3 shows what data exists at each stage of the protocol.

Receipt fields (100 B total)	Provider retains (ring buffer)	Revealed on audit
R_T (trace Merkle root)	INT8 inputs to all 7 matrices	Merkle openings from R_T
R_{KV} (KV Merkle root)	INT32 accumulator outputs	Merkle openings from R_{KV}
Manifest hash M	Requantized INT8 at each bridge	Opened intermediates at challenged positions
Token count N	`attn_out_i8` + quantization scales	Full prefix K, V at opened layers
	Per-tensor quantization scales	

Table 3: Data lifecycle: receipt (every response), retained state (RAM ring buffer, discarded after audit window), and audit openings (revealed only when challenged).

5 Security Analysis

This section maps concrete adversarial strategies to the protocol layers that detect them.

5.1 Model Swap or Downgrade

Proposition 1 (Model identity). *If the provider computes with weights $W' \neq W$ at any opened layer, the Freivalds check detects this with probability $\geq 1 - 2^{-32}$ per matrix.*

A single audit of a single token suffices. The guarantee is unconditional — it does not depend on statistical sampling or threshold calibration.

5.2 Attention Manipulation

Proposition 2 (Attention replay bound). *If attention replay with threshold τ accepts, the committed post-attention output agrees with the verifier's FP64 recomputation on at least fraction τ of INT8 elements. Disagreement is bounded by the FP16↔FP64 requantization corridor and the KV sampling boundary.*

The shell locks the inputs (Q, K, V) and outputs (post-W_o) of every attention block. Manipulation that alters the INT8 output beyond the corridor — local-window approximations, suppressed context, substituted patterns — produces element-wise disagreement exceeding τ and fails the replay. The remaining adversarial freedom is bounded by the corridor width and the KV sampling rate.

5.3 Fabricated Prefix Context

Proposition 3 (KV tampering detection). *If the provider tampered with m out of n prefix positions and the verifier samples k, detection probability is $P(\text{catch}) = 1 - (1 - m/n)^k$.*

Committed KV values cannot change after R_{KV} is sent (hash collision resistance). The verifier anchors the commitment to real computation by sampling earlier positions and running full shell verification. Detection probability increases monotonically with the tampering fraction m/n.

5.4 Deployment Configuration Tampering

The deployment manifest M binds the tokenizer, weight Merkle root, quantization scheme, sampling parameters, EOS

policy, and system prompt hash into the receipt. The verifier checks M against known-good values (e.g., the published tokenizer for a given model family, the weight Merkle root from the public checkpoint). Any change to the deployment configuration produces a different manifest.

6 The Attention Gap

The shell is exactly verifiable because its operations are deterministic or canonically recomputable: INT8 matmuls are checked cryptographically via Freivalds, while requantization, RoPE, SiLU, and RMSNorm are verified by exact or canonical recomputation. Attention (QK^T, softmax, αV) is computed in FP16/BF16, which is not bit-reproducible across hardware.

The protocol constrains the attention interior from both sides: inputs (Q, K, V) and outputs (post-W_o) are exactly verified by the shell. Attention replay directly checks consistency. Cross-layer constraints force fake attention to survive the residual stream.

The remaining adversarial freedom comes from two sources:

1. **FP16↔FP64 replay mismatch.** The verifier's FP64 reference and the GPU's FP16 computation produce slightly different results. The **requantization corridor** — the fraction of INT8 elements that disagree — bounds the adversary's room. This is an open empirical question. High agreement ($> 99\%$) means tight constraint; lower agreement means more room.
2. **Statistical KV anchoring.** The prefix K, V values used in attention replay are commitment-verified (they match R_{KV}) but only statistically anchored to real computation via sampled shell checks. Unsampled prefix positions are not independently verified, so the replay proves consistency with the *committed* prefix, not necessarily with the true execution at every earlier token.

7 Provider Costs

7.1 Storage

Only post-attention INT8 outputs and the associated per-tensor quantization scales require storage (everything else is re-derivable):

Model	Layers	hidden_dim	Per token
Llama 8B	32	4096	$\sim$ 128 KB
Llama 70B	80	8192	$\sim$ 640 KB
Llama 405B	126	16384	$\sim$ 2 MB

Table 4: Per-token storage for non-derivable intermediates

7.2 Audit Window

With a short audit window (1–2 minutes), traces fit in a RAM ring buffer with no disk I/O. For Llama 70B on 4× H100, write rate is $\sim$ 1.3 GB/s, requiring $\sim$ 315 GB ($\sim$ 16% of 2 TB system RAM) for a 2-minute window. Longer windows spill to NVMe or networked storage.

Short audit windows require automated auditing — the client's audit decision must be programmatic.

8 Related Work

The most direct alternative to VeriLM is proving inference in zero knowledge. Earlier verifiable-inference systems such as SafetyNets [1] and later zk approaches such as zkCNN [2] show that inference correctness can be proved by reducing neural networks to arithmetic-circuit-style verification. However, current ZK systems still impose large prover overheads. For a model that costs dollars per response to serve, this translates to hundreds or thousands of dollars per proved response — prohibitive for production inference.

Related cryptographic ML systems such as Delphi [6] address a different problem: privacy-preserving neural inference, where neither party should reveal its inputs or model. VeriLM instead targets model-provenance auditing for open weights, where the model is public and the goal is to verify that the claimed weights were actually used.

VeriLM makes the opposite tradeoff: verification is interactive (the provider must respond to challenges) and non-transferable (only the verifier who holds the key can check). In exchange, the normal serving path is unchanged and the per-response overhead is a 100-byte receipt. The expensive part — verification — only occurs on the small fraction of responses that are audited.

Trusted execution environments (TEEs) and remote attestation provide another alternative [3], [7]. TEEs can attest that specific code ran on specific hardware, avoiding the overhead of proof systems, but they introduce hardware trust assumptions, limit deployment flexibility, and tie verification to a specific vendor's attestation chain. VeriLM requires no hardware trust beyond standard computation.

9 Limitations and Extensions

The protocol's practical viability rests on two open empirical questions: the width of the requantization corridor (which bounds the attention gap) and whether storage and bandwidth costs are acceptable to providers at scale. Closing the attention gap entirely would require deterministic attention kernels or stronger proof systems, both of which violate the sidecar design constraint.

Two extensions could tighten the guarantees:

Score anchoring. The prover commits pre-softmax attention scores; the verifier spot-checks $\text{score}[t, j] \stackrel{?}{=} Q_t \cdot K_j$ as exact INT32 inner products. This constrains individual score entries independently of the output-level replay. The limitation is that the GPU computes softmax on FP16 scores, not the INT32 values the verifier checks, so the check cannot chain exactly to the softmax output.

Batch Freivalds on prefix. The verifier checks all prefix positions simultaneously using random linear combinations: pick random scalars $\alpha_1 ... \alpha_n$ and check $v \cdot (\sum \alpha_i x_i) \stackrel{?}{=} r^T \cdot (\sum \alpha_i z_i)$. One check covers all n positions with false-accept probability $\leq 1/2^{32}$. This upgrades KV provenance from statistical sampling to exact weight verification at all positions. Cost: $\sim 4 \times$ audit bandwidth, making it better suited as an optional deep audit mode.

10 Conclusion

VeriLM demonstrates that meaningful verification of LLM inference is possible without modifying the serving path or requiring expensive proof systems. The protocol's honest decomposition into exact, statistical, and approximate layers

allows clients to understand precisely what is and is not guaranteed. The 100-byte receipt imposes negligible overhead on normal serving; the full audit is CPU-feasible for the client.

The most important next step is empirical measurement of the requantization corridor, which will determine how tightly the protocol constrains attention manipulation in practice.

References

[1] Z. Ghodsi, T. Gu, and S. Garg, "SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud," in *Advances in Neural Information Processing Systems 30*, 2017. [Online]. Available: https://papers.nips.cc/paper/7053-safetynets-verifiable-execution-of-deep-neural-networks-on-an-untrusted-cloud

[2] T. Liu, X. Xie, and Y. Zhang, "zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2968–2985. doi: 10.1145/3460120.3485379.

[3] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation procedureS (RATS) Architecture," technical report RFC9334, Jan. 2023. doi: 10.17487/RFC9334.

[4] R. Freivalds, "Fast Probabilistic Algorithms," in *Mathematical Foundations of Computer Science 1979*, in Lecture Notes in Computer Science, vol. 74. Springer, 1979, pp. 57–69. doi: 10.1007/3-540-09526-8_5.

[5] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology – CRYPTO '87*, in Lecture Notes in Computer Science, vol. 293. Springer, 1987, pp. 369–378. doi: 10.1007/3-540-48184-2_32.

[6] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A Cryptographic Inference Service for Neural Networks," in *29th USENIX Security Symposium (USENIX Security 20)*, USENIX Association, 2020, pp. 2505–2522. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/mishra

[7] J. Ménétrey *et al.*, "Attestation Mechanisms for Trusted Execution Environments Demystified," in *Distributed Applications and Interoperable Systems*, in Lecture Notes in Computer Science, vol. 13272. Springer, 2022, pp. 95–113. doi: 10.1007/978-3-031-16092-9_7.